
Ottimizzazione su Reti

Capitolo 3

Alberi e grafi aciclici

Corso di Laurea Magistrale in Ingegneria Gestionale
Sapienza Università di Roma

Giampaolo Liuzzi
Dipartimento di Ingegneria Informatica, Automatica e Gestionale

Indice

1	Grafi aciclici, foreste e alberi	1
2	Caratterizzazioni equivalenti degli alberi	1
2.1	Conteggio degli spigoli in una foresta	2
3	Alberi ricoprenti	2
3.1	Costruzione incrementale	2
4	Scambio di spigoli e ciclo fondamentale	2
5	Alberi e matrice di incidenza	3
6	Alberi radicati e arborescenze	3
7	Grafi orientati aciclici	3
7.1	Algoritmo di Kahn	4
8	Implementazione con NetworkX	4
9	Registro delle notazioni	5
10	Errori frequenti	5
11	Esercizi	6
12	Sintesi del capitolo	6

Obiettivi di apprendimento

Al termine del capitolo lo studente dovrebbe essere in grado di:

- riconoscere alberi e foreste e usarne le caratterizzazioni equivalenti;
- costruire e verificare un albero ricoprente;
- comprendere il ruolo di un albero nella matrice di incidenza;
- distinguere alberi non orientati e arborescenze;
- riconoscere un grafo orientato aciclico e calcolarne un ordinamento topologico;
- implementare le operazioni fondamentali mediante NetworkX.

1 Grafi aciclici, foreste e alberi

In questa prima parte $G = (\mathcal{V}, \mathcal{E})$ è un grafo *non orientato*, finito e semplice.

Definizione 1.1 (Foresta e albero). *Un grafo che non contiene cicli è detto aciclico o foresta. Un grafo aciclico e connesso è detto albero.*

Le componenti connesse di una foresta sono alberi. Un singolo nodo, privo di spigoli, è dunque un albero. Questa convenzione evita eccezioni nelle proprietà ricorsive.

Definizione 1.2 (Foglia). *In un albero $T = (\mathcal{V}, \mathcal{E})$ con almeno due nodi, un nodo v di grado $d(v) = 1$ è detto foglia. Il suo unico vicino è talvolta detto padre quando si sceglie una radice.*

Proposizione 1.3. *Ogni albero con almeno due nodi possiede almeno due foglie.*

Idea della dimostrazione. Si consideri un cammino semplice di lunghezza massima. Se uno dei suoi estremi avesse un vicino non appartenente al cammino, lo si potrebbe prolungare; se avesse un altro vicino sul cammino, si formerebbe un ciclo. Entrambi gli estremi hanno quindi grado uno.

2 Caratterizzazioni equivalenti degli alberi

Teorema 2.1 (Proprietà fondamentali). *Per un grafo $G = (\mathcal{V}, \mathcal{E})$ con $n = |\mathcal{V}|$, le seguenti affermazioni sono equivalenti:*

1. G è un albero;
2. ogni coppia di nodi è collegata da un unico cammino;
3. G è aciclico e $|\mathcal{E}| = n - 1$;
4. G è connesso e $|\mathcal{E}| = n - 1$;
5. G è aciclico e l'aggiunta di uno spigolo tra due nodi non adiacenti crea un unico ciclo;
6. G è connesso e l'eliminazione di un qualsiasi spigolo lo rende non connesso.

Queste equivalenze esprimono due modi complementari di vedere un albero: è *massimalmente aciclico* e, nello stesso tempo, *minimamente connesso*.

Un controllo rapido

Un grafo con 8 nodi, 7 spigoli e connesso è necessariamente un albero. La sola uguaglianza $|\mathcal{E}| = |\mathcal{V}| - 1$, invece, non basta: un grafo può contenere un ciclo in una componente e avere un'altra componente isolata.

2.1 Conteggio degli spigoli in una foresta

Proposizione 2.2. *Se una foresta ha n nodi e k componenti connesse, allora*

$$|\mathcal{E}| = n - k.$$

In particolare, per un albero $k = 1$ e $|\mathcal{E}| = n - 1$.

3 Alberi ricoprenti

Definizione 3.1 (Albero ricoprente). *Dato un grafo connesso $G = (\mathcal{V}, \mathcal{E})$, un sottografo $T = (\mathcal{V}, F)$ è un albero ricoprente di G se $F \subseteq \mathcal{E}$ e T è un albero.*

L'aggettivo “ricoprente” indica che T contiene tutti i nodi di G , ma soltanto $n - 1$ dei suoi spigoli. Un grafo possiede un albero ricoprente se e solo se è connesso.

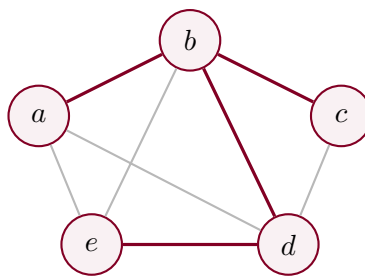


Figura 1: In rosso un albero ricoprente: collega tutti i cinque nodi con quattro spigoli.

3.1 Costruzione incrementale

Un albero ricoprente può essere costruito sfruttando i tagli introdotti nel Capitolo 2.

1. Scegliere $r \in \mathcal{V}$ e porre $S = \{r\}$, $F = \emptyset$.
2. Finché $S \neq \mathcal{V}$, scegliere uno spigolo $e = \{u, v\} \in \delta(S)$ con $u \in S$ e $v \notin S$.
3. Porre $S \leftarrow S \cup \{v\}$ e $F \leftarrow F \cup \{e\}$.

Se a un certo passo $\delta(S) = \emptyset$, il grafo non è connesso. Altrimenti, ogni iterazione aggiunge un nodo e uno spigolo senza creare cicli; al termine $T = (\mathcal{V}, F)$ è un albero ricoprente.

Attenzione

L'algoritmo non specifica quale spigolo del taglio scegliere. Scelte diverse possono produrre alberi ricoprenti diversi. Se agli spigoli sono associati costi e si sceglie sempre uno spigolo di costo minimo ammissibile, si entra nel problema dell'albero ricoprente di costo minimo.

4 Scambio di spigoli e ciclo fondamentale

Sia $T = (\mathcal{V}, F)$ un albero ricoprente di G e sia $e = \{u, v\} \in \mathcal{E} \setminus F$. In T esiste un unico cammino tra u e v ; aggiungendo e si crea quindi un unico ciclo, detto *ciclo fondamentale* di e rispetto a T .

Se dal ciclo fondamentale si elimina uno spigolo $f \neq e$, si ottiene

$$F' = F \cup \{e\} \setminus \{f\},$$

e $(\mathcal{V}, F')$ è ancora un albero ricoprente. Questa operazione di scambio sarà essenziale nel simplesso su reti: un arco entra nella base, si crea un ciclo e un altro arco esce.

5 Alberi e matrice di incidenza

Sia $G = (\mathcal{V}, \mathcal{E})$ un grafo orientato ottenuto assegnando un verso arbitrario agli spigoli di un grafo connesso. Manteniamo la convenzione fissata:

$$A_{ie} = \begin{cases} -1, & \text{se il nodo } i \text{ è l'origine dell'arco } e, \\ +1, & \text{se il nodo } i \text{ è la destinazione dell'arco } e, \\ 0, & \text{altrimenti.} \end{cases}$$

Teorema 5.1 (Rango della matrice di incidenza). *Se G è connesso e $|\mathcal{V}| = n$, allora $\text{rank}(A) = n - 1$. Eliminando una riga qualsiasi di A si ottiene la matrice ridotta $\bar{A}$; un insieme F di $n - 1$ colonne di $\bar{A}$ è linearmente indipendente se e solo se gli archi di F , considerati senza orientamento, formano un albero ricoprente.*

L'orientamento scelto modifica soltanto il segno di alcune colonne e non l'indipendenza lineare. Il teorema stabilisce il ponte tra la struttura combinatoria degli alberi e le basi algebriche impiegate nel Capitolo 4.

Matrice ridotta di un albero

Per gli archi $(1, 2)$, $(1, 3)$ e $(3, 4)$, eliminando la riga del nodo 1 si ottiene

$$\bar{A} = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & -1 \\ 0 & 0 & 1 \end{pmatrix}, \quad \det(\bar{A}) = 1.$$

Le tre colonne corrispondono a un albero sui quattro nodi e formano una base.

6 Alberi radicati e arborescenze

Un albero non orientato diventa *radicato* quando si distingue un nodo r , detto radice. La distanza dalla radice definisce i livelli; ogni nodo diverso da r ha un unico padre e può avere zero o più figli.

Definizione 6.1 (Arborescenza uscente). *Un digrafo $T = (\mathcal{V}, F)$ è un'arborescenza uscente radicata in r se, per ogni $v \in \mathcal{V} \setminus \{r\}$, esiste un unico cammino orientato da r a v .*

In un'arborescenza uscente, $d^-(r) = 0$ e $d^-(v) = 1$ per ogni $v \neq r$. In modo simmetrico, in un'arborescenza *entrante* esiste un unico cammino orientato da ogni nodo verso r ; quindi $d^+(r) = 0$ e $d^+(v) = 1$ per $v \neq r$.

Attenzione

Un orientamento aciclico di un albero non è necessariamente un'arborescenza: può avere più sorgenti oppure un nodo non raggiungibile dalla radice prescelta. Occorre verificare anche la proprietà di raggiungibilità orientata.

7 Grafi orientati aciclici

Definizione 7.1 (DAG). *Un grafo orientato privo di cicli orientati è detto grafo orientato aciclico, o DAG (directed acyclic graph).*

Un DAG può contenere un ciclo nel grafo non orientato sottostante: ciò che è vietato è un ciclo percorribile rispettando il verso di tutti gli archi.

Definizione 7.2 (Ordinamento topologico). *Un ordinamento $v_1, \dots, v_n$ dei nodi di un digrafo è topologico se, per ogni arco $(v_i, v_j) \in \mathcal{E}$, si ha $i < j$.*

Teorema 7.3. *Un digrafo ammette un ordinamento topologico se e solo se è aciclico.*

Dimostrazione. Se esiste un ordinamento topologico, lungo ogni arco l'indice cresce; non è quindi possibile tornare al nodo di partenza. Viceversa, ogni DAG non vuoto possiede almeno una sorgente. Eliminando iterativamente una sorgente e i suoi archi uscenti si ottiene un ordinamento topologico.

7.1 Algoritmo di Kahn

1. Calcolare il grado entrante di ogni nodo e raccogliere i nodi con $d^-(v) = 0$.
2. Estrarre una sorgente v , aggiungerla all'ordinamento ed eliminare i suoi archi uscenti.
3. Inserire tra le sorgenti ogni nodo il cui grado entrante diventa zero.
4. Se sono stati ordinati tutti i nodi, il grafo è aciclico; altrimenti gli archi residui contengono un ciclo.

Con liste di adiacenza, il costo è $O(n + m)$. Se più sorgenti sono disponibili, l'ordinamento topologico non è necessariamente unico.

Relazioni di precedenza

Siano presenti gli archi (A, C) , (B, C) , (B, D) , (C, E) e (D, E) . Due ordinamenti topologici sono

$$A, B, C, D, E \quad \text{e} \quad B, A, D, C, E.$$

Le attività A e B possono essere scambiate perché nessun vincolo di precedenza le ordina tra loro.

8 Implementazione con NetworkX

Python/NetworkX

In NetworkX le funzioni principali sono:

Operazione	Funzione
verifica di un albero	<code>nx.is_tree(G)</code>
verifica di una foresta	<code>nx.is_forest(G)</code>
albero ricoprente	<code>nx.minimum_spanning_tree(G)</code>
verifica di un DAG	<code>nx.is_directed_acyclic_graph(D)</code>
ordinamento topologico	<code>nx.topological_sort(D)</code>
arborescenza	<code>nx.is_arborescence(D)</code>

Listing 1: Albero ricoprente e scambio fondamentale

```
import networkx as nx

G = nx.Graph()
G.add_weighted_edges_from([
    (1, 2, 4), (1, 3, 2), (2, 3, 1),
    (2, 4, 5), (3, 4, 3)
```

```

])

T = nx.minimum_spanning_tree(G, weight="weight")
print(sorted(T.edges(data="weight")))
print(nx.is_tree(T), T.number_of_edges())

# Il cammino in T chiuso dall'arco non di albero (1, 2)
print(nx.shortest_path(T, 1, 2))

```

Listing 2: Riconoscimento di un DAG e ordinamento topologico

```

D = nx.DiGraph([
    ("A", "C"), ("B", "C"), ("B", "D"),
    ("C", "E"), ("D", "E")
])

if nx.is_directed_acyclic_graph(D):
    ordine = list(nx.topological_sort(D))
    print(ordine)
else:
    print("Il digrafo contiene un ciclo orientato")

```

Attenzione

`minimum_spanning_tree` usa l'attributo `weight` e risolve un problema di costo minimo. Per ottenere un qualunque albero ricoprente si può invece eseguire una visita DFS o BFS e conservare gli spigoli di scoperta.

9 Registro delle notazioni

Registro delle notazioni

Simbolo	Significato
$G = (\mathcal{V}, \mathcal{E})$	grafo, orientato o non orientato secondo il contesto
$T = (\mathcal{V}, F)$	albero ricoprente, con $F \subseteq \mathcal{E}$
n, m	numero di nodi e di spigoli/archi
$\delta(S)$	taglio non orientato indotto da $S \subset \mathcal{V}$
$d(v)$	grado di v in un grafo non orientato
$\delta^-(v), \delta^+(v)$	archi entranti e uscenti da v
$d^-(v), d^+(v)$	grado entrante e grado uscente
A	matrice di incidenza orientata: -1 in origine, $+1$ a destinazione
$\bar{A}$	matrice di incidenza con una riga eliminata

10 Errori frequenti

1. Concludere che un grafo è un albero soltanto perché ha $n - 1$ spigoli.
2. Confondere un sottografo ricoprente con un albero ricoprente: serve anche connessione e assenza di cicli.
3. Credere che l'orientamento degli archi influenzi l'indipendenza delle colonne di incidenza.
4. Confondere un DAG con un albero: un DAG può avere molti più di $n - 1$ archi.

5. Verificare l'assenza di cicli nel grafo sottostante invece che di cicli orientati.
6. Confondere un ordinamento topologico con l'ordine numerico originario delle etichette.
7. Usare `minimum_spanning_tree` su un digrafo senza chiarire il problema orientato da risolvere.

11 Esercizi

Esercizio 3.1 – Caratterizzazioni. Per ciascuna coppia di proprietà seguente, stabilire se identifica necessariamente un albero e motivare: (a) connesso e aciclico; (b) $n - 1$ spigoli; (c) connesso e $n - 1$ spigoli; (d) aciclico e $n - 1$ spigoli.

Esercizio 3.2 – Costruzione. Dato G con $\mathcal{V} = \{1, 2, 3, 4, 5, 6\}$ ed

$$\mathcal{E} = \{\{1, 2\}, \{1, 3\}, \{2, 3\}, \{2, 4\}, \{3, 5\}, \{4, 5\}, \{4, 6\}, \{5, 6\}\},$$

costruire due alberi ricoprenti distinti mediante la procedura incrementale.

Esercizio 3.3 – Ciclo fondamentale. Scelto uno degli alberi dell'Esercizio 3.2, aggiungere uno spigolo non appartenente all'albero, determinare il ciclo fondamentale e produrre un nuovo albero mediante uno scambio.

Esercizio 3.4 – Incidenza. Orientare arbitrariamente l'albero dell'Esercizio 3.2, costruire A , eliminare la riga del nodo 6 e verificare che $\det(\bar{A}) = \pm 1$.

Esercizio 3.5 – DAG. Per il digrafo con archi

$$(1, 3), (1, 4), (2, 4), (2, 5), (3, 5), (4, 5), (4, 6), (5, 6),$$

trovare due ordinamenti topologici distinti. Aggiungere poi un solo arco che produca un ciclo orientato.

Esercizio 3.6 – NetworkX. Implementare gli Esercizi 3.2 e 3.5. Verificare automaticamente gli alberi ricoprenti ed enumerare almeno tre ordinamenti topologici mediante `nx.all_topological_sorts`. Confrontarli con quelli trovati a mano.

12 Sintesi del capitolo

Obiettivi di apprendimento

- Un albero è un grafo connesso e aciclico e possiede esattamente $n - 1$ spigoli.
- Un grafo è connesso se e solo se possiede un albero ricoprente.
- L'aggiunta di uno spigolo a un albero genera un unico ciclo fondamentale.
- Le basi della matrice di incidenza ridotta corrispondono agli alberi ricoprenti.
- Un digrafo è aciclico se e solo se ammette un ordinamento topologico.
- Arborescenze, alberi e DAG sono strutture diverse e non vanno confuse.

Verso il capitolo successivo. Nel Capitolo 4 gli archi saranno dotati di capacità, costi e flussi. Gli alberi ricoprenti diventeranno basi del simpleso su reti, mentre la convenzione -1 all'origine e $+1$ alla destinazione consentirà di scrivere i bilanci nella forma $Ax = b$, coerente con NetworkX.